

02

System Architecture & Data Design

How TrustHome PH is put together: the hybrid "Firebase Spark + free trusted API" architecture, the decisions behind it, the Firestore data model, the API contract, and how each core algorithm works.

Revision	Rev 2.0 — 1 October 2026
Supersedes	System Build Plan (Firebase Edition)
Stack baseline	React + Firebase Spark (Auth, Firestore, Hosting) + Cloudinary Free + Vercel Hobby API + GitHub Actions. No paid plan, no credit card.
Audience	Developers (all five), AI coding assistants, technical panel questions
Team	Andresa (Scrum Master) · Carillo · Laurente · Prades · Solpico

READ THIS FIRST

The original plan assumed Cloud Functions, but on Firebase they need the Blaze plan, and nobody on the team can open one. Rev 2 keeps everything that already works on the free Spark plan (Auth, Firestore, Hosting, security rules) and Cloudinary for files. It adds one small piece: a free trusted API on Vercel Hobby (Node.js, no credit card). That API handles the three jobs a browser must never do itself. It confirms bookings without overlaps (F-05). It signs private document uploads, and it issues short-lived document links (F-16). Scheduled work stays on GitHub Actions. The API contract is framework-neutral, so a future Laravel backend could replace it without touching the frontend.

CONTENTS

1. Architecture decisions (ADRs)	2. Technology stack (current)	4. Repository layout	7. Core algorithms
	3. System overview	5. Firestore data model	8. Notifications design
		6. Trusted API contract	9. Coding standards (all members, all AI output)

Document set: 01 Product Requirements & Scope · **02 System Architecture & Data Design** · 03 Infrastructure, Setup & Deployment · 04 Security & Privacy · 05 UI/UX Design System · 06 Roadmap & Release Plan · 07 QA & Testing Strategy

1. Architecture decisions (ADRs)

Architecture Decision Records capture *why* the system looks the way it does. When a panelist asks "why didn't you just...?", the answer is here.

ADR-001: No dependency on Firebase Blaze

Context. Cloud Functions, Cloud Storage for Firebase (since September 2024), managed Firestore export/import, and Cloud Build all require the Blaze pay-as-you-go plan. Blaze needs a billing account with a card that Google accepts. The team tried, and deployment of the `confirmBooking` callable failed on 1 Oct 2026 because Cloud Build requires Blaze. Prepaid virtual cards such as GCash's are not reliably accepted.

Decision. Treat "no credit card" as a hard non-functional requirement (NFR-COST-01). Use Firebase only for the parts that are free on Spark: **Authentication, Cloud Firestore, Hosting, Security Rules.**

Consequences. Anything that needs trusted server code, a secret, or a schedule runs **outside Firebase**: the trusted API on Vercel, and scheduled jobs on GitHub Actions. The `functions/` folder stays in the repo as reference code but is **not deployed**.

ADR-002: Add a free trusted API (Vercel Hobby, Node.js)

Context. Three operations cannot safely run in the browser:

- 1. **Confirming a booking.** Client-side transactions can be bypassed by a modified client, and owners must be blocked from writing `Confirmed` directly (F-05).
- 2. **Signing Cloudinary uploads.** Signing requires the API secret, which must never ship to the browser.
- 3. **Issuing document links.** A short-lived link to a private ID or title can only be created with the API secret (F-16).

Options considered.

Option	Card needed?	Effort (< 2 weeks)	Verdict
Firebase Blaze + Cloud Functions	Yes	Lowest (code exists)	Blocked
Vercel Hobby serverless functions (Node.js)	No	Low: same JS, <code>firebase-admin</code> works unchanged	Chosen
Render free web service	Varies; spins down when idle (slow first request)	Low-medium	Fallback host for the same code
Cloudflare Workers	No	High: <code>firebase-admin</code> does not run there; needs REST + manual JWT	Rejected for now
Laravel API on free PHP hosting	No	High: rewrite plus hosting limits on free PHP hosts	Post-defense option (ADR-003)
Move to Supabase / another BaaS	No	Very high: full data migration	Rejected

Decision. Build a small Node.js API in `api-server/`, deployed to Vercel Hobby in region `sin1` (Singapore, next to Firestore `asia-southeast1`). It uses the Firebase Admin SDK (bypasses rules, so it must do its own checks) and the Cloudinary Node SDK.

Consequences. It adds one deploy target and one set of secrets. Vercel Hobby is for non-commercial use, which fits a capstone; a commercial launch would move to a paid plan or to the Laravel API. Cold starts add about 1 s to the first request after idle time.

ADR-003: Keep the API contract framework-neutral (Laravel-ready)

The team has noted that Laravel offers more out of the box: ORM, migrations, queues, scheduler, policies, mail. That is true. A full rewrite is not realistic before the defense, but it can be prepared for:

- The frontend talks to the API only through `src/services/api.js`, using plain REST/JSON and a `Bearer` Firebase ID token.
- Error responses use one shape: `{ "error": { "code": "BOOKING_OVERLAP", "message": "..." } }`.
- Business rules live in this document and in *01 Requirements*, not only in code.

Post-defense migration path. (1) Re-implement the same endpoints in Laravel, verifying Firebase ID tokens with the `kreatit/firebase-php` package. (2) Switch `VITE_API_BASE_URL`. (3) Move collections to MySQL one at a time, starting with bookings and disputes, where relational constraints help most. (4) Replace Firebase Auth last. Each step is reversible.

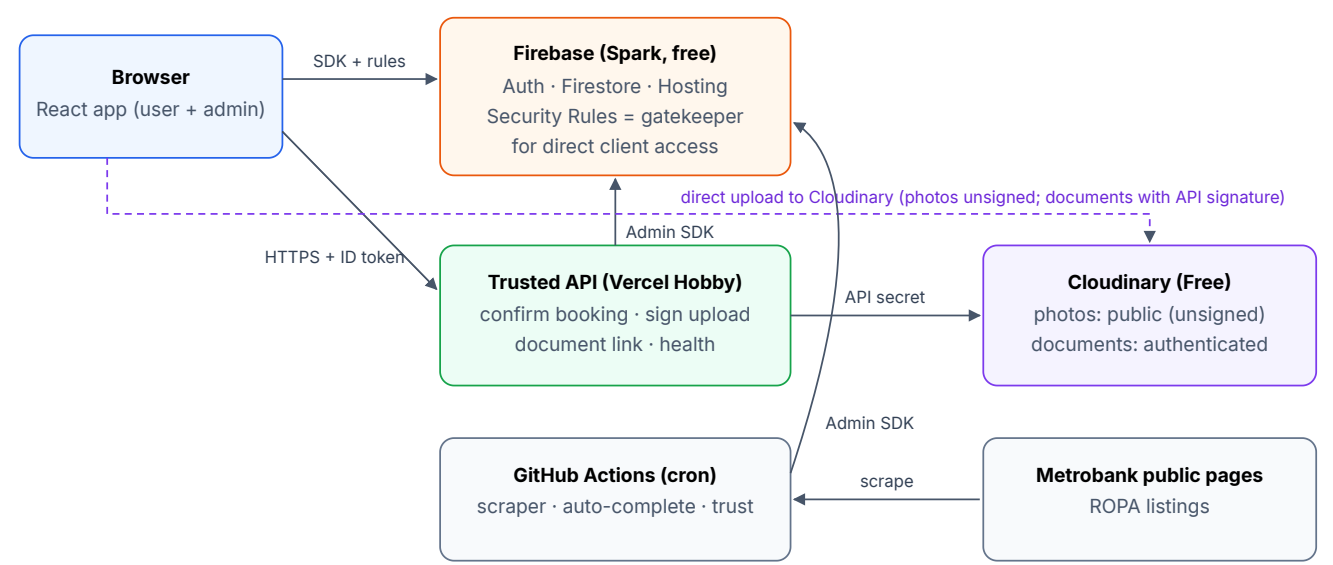
SAY THIS TO THE PANEL · "We chose Firebase for speed. When the free tier stopped covering trusted server code, we added a minimal API instead of paying or weakening security. The API's contract is independent of Firebase, so the backend can evolve, for example to Laravel, without rewriting the app."

2. Technology stack (current)

Layer	Technology	Plan / cost	Notes
Web app	React 18 + Vite, React Router, lazy routes	—	Mobile-first user pages, desktop admin
Auth	Firebase Authentication (email/password, Google)	Spark, free	ID tokens verified by the API
Database	Cloud Firestore, <code>asia-southeast1</code>	Spark: 1 GiB, 50K reads / 20K writes / 20K deletes per day	Rules deny by default
Static hosting	Firebase Hosting (<code>trusthome-ph.web.app</code>)	Spark: 10 GB stored, 360 MB/day transfer	Deployed Oct 1
Trusted API	Vercel Hobby, Node.js 22 serverless functions	Free, no card: 1M invocations, 4 h active CPU, 100 GB transfer per month	New in v1.4
Files	Cloudinary	Free plan (monthly credits)	Photos: unsigned preset. Documents: signed, <code>authenticated</code>
Scheduled jobs	GitHub Actions (Python 3.11 + <code>firebase-admin</code>)	Free	Scraper, auto-complete, trust recompute
Maps	Leaflet + OpenStreetMap tiles; Nominatim geocoding	Free, attribution required	Respect the Nominatim usage policy (≤ 1 req/s)
Tests	<code>node --test</code> , Python <code>unittest</code> , Firebase Emulator Suite	—	47 Node + 23 Python tests passing (Oct 1)

WATCH OUT · Node version. The old `functions/` package targets Node 20, which reached end-of-life in April 2026. Use **Node 22 LTS** for `api-server/`, and pin it in `package.json` (`"engines": { "node": "22.x" }`) so local, CI and Vercel agree.

3. System overview



3.1 Trust boundaries: who may write what

Use this table to decide where a feature belongs. **If a write needs a secret, must be atomic across documents the user does not own, or decides money, time or trust, it does not go in the browser.**

Operation	Runs in	Enforced by
Register, edit own profile (not role/status)	Browser	Firestore rules
Create/edit own listing (pending), upload photos	Browser	Rules (field allowlist) + unsigned preset limits
Upload verification documents	Browser → Cloudinary with API signature	API <code>POST /api/uploads/sign</code>
Admin approve / request changes / reject	Browser (admin)	Rules (<code>isAdmin()</code>)
Request a booking, withdraw, decline	Browser	Rules (state + participant checks)
Confirm a booking	Trusted API only	Admin SDK transaction; rules deny client <code>Confirmed</code>
Log deposit reference, complete, rate, dispute	Browser	Rules
Resolve dispute, suspend user	Browser (admin)	Rules
View a verification document	Trusted API issues a 5-minute link	API role check + Cloudinary signature
Auto-complete expired bookings, trust scores, scrape	GitHub Actions	Service account; rules deny clients

BEST PRACTICE · Defense in depth. The UI hides buttons, the rules deny forbidden writes, and the API re-checks everything. Each layer assumes the one in front of it has been bypassed. A security test only counts if it goes **around** the UI, through a direct SDK write or a raw HTTP call.

4. Repository layout

```
trusthome-ph/
├── src/                React app
│   ├── services/       firebase.js, api.js (all API calls), uploads.js
│   ├── features/       listings/, bookings/, disputes/, admin/, notifications/ ...
│   ├── components/     shared UI (Button, Badge, Card, Field...)
│   └── lib/             pure logic + unit tests (validation, overlap, trust, loan)
├── api-server/         NEW - Vercel project (Root Directory = api-server)
│   ├── api/            one file per endpoint (file-based routing)
│   ├── lib/            auth.js, cors.js, errors.js, firebaseAdmin.js, cloudinary.js, bookings.js
│   ├── test/           node --test against the Firestore emulator
│   └── vercel.json
├── functions/          legacy callable - reference only, NOT deployed
├── bank_scraper/       Python: scraper, jobs, migrations (dry-run first), seeders
├── firestore.rules · firestore.indexes.json · firebase.json
└── .github/workflows/  scrape.yml (+ ci.yml recommended)
```

BEST PRACTICE · Keep business logic in **pure functions** (`src/lib/overlap.js` , `api-server/lib/bookings.js`) that take plain data and return results. They can be unit-tested without Firebase, reused by the API and the client, and ported to PHP line by line if Laravel ever happens.

5. Firestore data model

Firestore has collections of JSON-like documents, no joins and no foreign keys. Relationships are document IDs stored as fields. **Denormalize for reads, but keep one owner of truth per fact.**

Collection	Key fields	Who reads	Who writes
<code>users/{uid}</code>	<code>name</code> , <code>email</code> , <code>role</code> (user/admin), <code>status</code> (active/suspended), <code>createdAt</code>	Owner; public subset via profile	Owner (not role/status); admin
<code>listings/{id}</code>	<code>ownerId</code> , <code>title</code> , <code>description</code> , <code>purpose</code> (rent/sale), <code>pricePeriod</code> (nightly/monthly/once), <code>type</code> , <code>city</code> , <code>area</code> , <code>bedrooms</code> , <code>bathrooms</code> , <code>floorArea</code> , <code>price</code> , <code>amenities[]</code> , <code>photoUrls[]</code> , <code>showingWindows[]</code> , <code>geo</code> { <code>geohash</code> (5), <code>approximate:true</code> }, <code>verificationStatus</code> (pending/verified/changes_requested/rejected), <code>reviewNote</code> , <code>verifiedAt</code> , <code>createdAt</code> , <code>updatedAt</code>	Public if verified and owner active; owner; admin	Owner (limited fields); admin (status)
<code>listingPrivate/{listingId}</code>	<code>address</code> , <code>documents</code> { <code>ownership</code> { <code>publicId</code> , <code>format</code> , <code>resourceType</code> }, <code>govId</code> {...}}, <code>legacy URL fields</code> (until migrated)	Owner, admin, API	Owner on create/edit; API; migration
<code>bookings/{id}</code>	<code>listingId</code> , <code>ownerId</code> , <code>renterId</code> , <code>startDate</code> , <code>endDate</code> (Timestamp), <code>status</code> , <code>priorStatus</code> , <code>disputeId</code> , <code>depositReference</code> , <code>confirmedAt</code> , <code>confirmedBy</code> , <code>createdAt</code>	Renter, owner, admin	Renter (create), API (confirm), rules-checked transitions
<code>bookingLocks/{listingId} *</code>	<code>version</code> , <code>updatedAt</code>	API only	API only (transaction)

Collection	Key fields	Who reads	Who writes
<code>ratings/{id}</code>	<code>bookingId, listingId, raterId, rateeId, score 1-5, comment, createdAt</code>	Public	Participant of a Completed booking, once
<code>disputes/{id}</code>	<code>bookingId, raisedBy, reason, evidence, status (Open/Founded/Dismissed), resolutionNotes, resolvedBy, resolvedAt</code>	Reporter, owner, admin	Renter (create), admin (resolve)
<code>trustScores/{listingId}</code>	<code>score, completedBookingsCount, avgRating, priceFairnessLabel, manualReviewRequired, selfBookingPatternCount, computedAt</code>	Public	GitHub Actions only
<code>notifications/{id}</code>	<code>recipientId (uid or "__admins__"), type, title, link, read, createdAt</code>	Recipient (admins for <code>__admins__</code>)	Creators allowed by type (rules)
<code>bankProperties/{bank_ref}</code>	<code>bank, referenceNo, title, location, price, floorArea, lotArea, listingUrl, imageUrl, status, firstSeen, lastSeen</code>	Public	Scraper only
<code>scrapeJobs/{id}</code>	<code>bank, requestedBy, status (queued/running/done/failed), log, timestamps</code>	Admin	Admin (queue), worker (update)
<code>savedSearches/{id}</code>	Retired; kept read-only	Owner	Nobody

* Use the lock collection name that already exists in the callable's source. The name shown here is illustrative.

BEST PRACTICE · Data-model rules of thumb.

- Store dates as Firestore `Timestamp`, never strings. Convert to Asia/Manila only in the UI.
- Store money as **whole pesos (integers)**, never floats.
- Copy `ownerId` onto bookings so rules can authorize without an extra `get()`.
- Public documents must never contain private data. Field-level privacy does not exist in Firestore: if a document is readable, every field in it is readable.
- Every list query needs an index. Keep `firestore.indexes.json` in Git and deploy it with the rules.

Required composite indexes (minimum): `bookings (listingId ASC, status ASC)`, `bookings (renterId ASC, createdAt DESC)`, `bookings (ownerId ASC, status ASC)`, `notifications (recipientId ASC, createdAt DESC)`, `listings (verificationStatus ASC, city ASC, price ASC)`.

6. Trusted API contract

Base URL: `https://<project>.vercel.app/api` (stored in the frontend as `VITE_API_BASE_URL`). **Auth:** Authorization: Bearer `<Firebase ID token>` on every endpoint except `/health`. **Format:** JSON in, JSON out. **Errors:** `{ "error": { "code": "STRING_CODE", "message": "Human-readable" } }`.

Method & path	Who	Purpose	Success	Errors
GET /api/health	Anyone	Uptime and version check	200 {status, version, region}	—
POST /api/bookings/{id}/confirm	Listing owner	Confirm a Pending booking atomically	200 {status:"Confirmed"}	401, 403 NOT_OWNER / ACCOUNT_SUSPENDED , 404, 409 BOOKING_OVERLAP / NOT_PENDING , 500
POST /api/uploads/sign	Signed-in active user	Signature for a private document upload	200 {signature, timestamp, apiKey, cloudName, params}	401, 403, 422 BAD_KIND
GET /api/listings/{id}/documents/{kind}	Listing owner or admin	Short-lived link to ownership or govId	200 {url, expiresAt}	401, 403, 404 NO_DOCUMENT

BEST PRACTICE · API best practices applied: verify the token on every call; **re-read role and status from Firestore** (never trust claims sent by the client); validate the body against a schema; return the *minimum* data; use correct status codes (409 for business conflicts, 422 for invalid input); make confirm **idempotent** (confirming an already-Confirmed booking by the same owner returns 200 and does nothing); allow-list CORS origins; never log tokens, document URLs or addresses.

6.1 Shared helpers

```
// api-server/lib/firebaseAdmin.js
import { initializeApp, cert, getApps } from 'firebase-admin/app';
import { getAuth } from 'firebase-admin/auth';
import { getFirestore } from 'firebase-admin/firestore';

if (!getApps().length) {
  initializeApp({ credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
    privateKey: process.env.FIREBASE_PRIVATE_KEY.replace(/\\n/g, '\n'),
  }) });
}
export const adminAuth = getAuth();
export const db = getFirestore();
```

```
// api-server/lib/auth.js
import { adminAuth, db } from './firebaseAdmin.js';
import { HttpError } from './errors.js';

export async function requireUser(req, { checkRevoked = false } = {}) {
  const m = /^Bearer (.+)\$/i.exec(req.headers.authorization || '');
  if (!m) throw new HttpError(401, 'NO_TOKEN', 'Sign in again.');
```

let decoded;

```
try { decoded = await adminAuth.verifyIdToken(m[1], checkRevoked); }
catch { throw new HttpError(401, 'BAD_TOKEN', 'Your session expired. Sign in again.');
```

const snap = await db.doc(`users/\${decoded.uid}`).get();

```
const profile = snap.data();
if (!profile) throw new HttpError(403, 'NO_PROFILE', 'Account profile not found.');
```

if (profile.status !== 'active') throw new HttpError(403, 'ACCOUNT_SUSPENDED', 'This account is suspended.');

```
return { uid: decoded.uid, role: profile.role, profile };
}
```

```
// api-server/lib/http.js - one wrapper for CORS, method check and error shape
const ALLOWED = (process.env.ALLOWED_ORIGINS || '').split(',');
```

export function handler(methods, fn) {

```
  return async (req, res) => {
    const origin = req.headers.origin;
    if (ALLOWED.includes(origin)) {
      res.setHeader('Access-Control-Allow-Origin', origin);
      res.setHeader('Vary', 'Origin');
```

res.setHeader('Access-Control-Allow-Headers', 'Authorization, Content-Type');

```
      res.setHeader('Access-Control-Allow-Methods', methods.join(', '));
    }
    if (req.method === 'OPTIONS') return res.status(204).end();
    if (!methods.includes(req.method)) return res.status(405).json({ error: { code: 'METHOD', message: 'Not
allowed' } });
    try { return res.status(200).json(await fn(req)); }
    catch (e) {
      const status = e.status || 500;
      if (status === 500) console.error('[api]', e.message); // no tokens / PII in logs
      return res.status(status).json({ error: { code: e.code || 'INTERNAL', message: status === 500 ?
'Something went wrong.' : e.message } });
    }
  };
}
```

6.2 Booking confirmation (closes F-05)

```
// api-server/api/bookings/[id]/confirm.js
import { handler } from '../../lib/http.js';
import { requireUser } from '../../lib/auth.js';
import { db } from '../../lib/firebaseAdmin.js';
import { FieldValue } from 'firebase-admin/firestore';
import { HttpError } from '../../lib/errors.js';
import { overlaps } from '../../lib/bookings.js';

export default handler(['POST', 'OPTIONS'], async (req) => {
  const { uid } = await requireUser(req);
  const bookingRef = db.doc(`bookings/${req.query.id}`);

  return db.runTransaction(async (tx) => {
    const b = await tx.get(bookingRef);
    if (!b.exists) throw new HttpError(404, 'NOT_FOUND', 'Booking not found.');
```

const booking = b.data();

if (booking.ownerId !== uid) throw new HttpError(403, 'NOT_OWNER', 'Only the listing owner can confirm.');

if (booking.status === 'Confirmed') return { status: 'Confirmed', idempotent: true };

if (booking.status !== 'Pending') throw new HttpError(409, 'NOT_PENDING', 'This request is no longer pending.');

const lockRef = db.doc(`bookingLocks/\${booking.listingId}`);

const lock = await tx.get(lockRef); // every confirm for this listing reads + writes this doc

const taken = await tx.get(db.collection('bookings').where('listingId', '=', booking.listingId).where('status', 'in', ['Confirmed', 'Disputed']));

const clash = taken.docs.some(d => d.id !== b.id && overlaps(d.data(), booking));

if (clash) throw new HttpError(409, 'BOOKING_OVERLAP', 'These dates are already confirmed for another renter.');

tx.update(bookingRef, { status: 'Confirmed', confirmedAt: FieldValue.serverTimestamp(), confirmedBy: uid });

tx.set(lockRef, { version: (lock.data()?.version ?? 0) + 1, updatedAt: FieldValue.serverTimestamp() }, { merge: true });

tx.create(db.collection('notifications').doc(), {

 recipientId: booking.renterId, type: 'booking_confirmed', link: `/activity?tab=bookings`,

 title: 'Your booking was confirmed', read: false, createdAt: FieldValue.serverTimestamp() });

// + address handoff write, as already implemented in the legacy callable

return { status: 'Confirmed' };

});

```
// api-server/lib/bookings.js - pure, unit-tested, shared with the client pre-check
export const overlaps = (a, b) =>
  a.startDate.toMillis() < b.endDate.toMillis() && b.startDate.toMillis() < a.endDate.toMillis(); // half-open [start, end)
```

Why this is race-safe. Server-side Firestore transactions lock the documents they read. Two owners' tabs confirming two overlapping requests both read and write the same `bookingLocks/{listingId}` document, so Firestore forces them to run one after the other. The second one re-runs, sees the first booking as Confirmed, and fails with 409. The emulator already proved this for the legacy callable. TC-API-006 repeats it against the live API.

Notification inside the transaction? Here it is included, because the API is trusted and the write is cheap. Client-created notifications stay **best-effort** (NFR-REL-01).

6.3 Private documents (closes F-16)

```
// api-server/api/uploads/sign.js - browser asks for a signature, then uploads straight to Cloudinary
import { v2 as cloudinary } from 'cloudinary';
export default handler(['POST', 'OPTIONS'], async (req) => {
  const { uid } = await requireUser(req);
  const { listingId, kind } = req.body || {};
  if (!['ownership', 'govId'].includes(kind)) throw new HttpError(422, 'BAD_KIND', 'Unknown document type.');
```

const timestamp = Math.round(Date.now() / 1000);

const params = { timestamp, upload_preset: 'trustrhome_private_docs', type: 'authenticated', asset_folder: `trustrhome/private/\${uid}`,

tags: `verification,\${kind}`, context: `listing=\${listingId}` };

const signature = cloudinary.utils.api_sign_request(params, process.env.CLOUDINARY_API_SECRET);

return { signature, timestamp, apiKey: process.env.CLOUDINARY_API_KEY,

cloudName: process.env.CLOUDINARY_CLOUD_NAME, params };

});

```
// api-server/api/listings/[id]/documents/[kind].js - 5-minute link for owner or admin only
export default handler(['GET', 'OPTIONS'], async (req) => {
  const { uid, role } = await requireUser(req, { checkRevoked: true });
  const listing = (await db.doc(`listings/${req.query.id}`).get()).data();
  if (!listing) throw new HttpError(404, 'NOT_FOUND', 'Listing not found.');
```

if (role !== 'admin' && listing.ownerId !== uid) throw new HttpError(403, 'FORBIDDEN', 'Not allowed.');

const doc = (await db.doc(`listingPrivate/\${req.query.id}`).get()).data()?.documents?.[req.query.kind];

if (!doc) throw new HttpError(404, 'NO_DOCUMENT', 'No document on file.');

const expiresAt = Math.round(Date.now() / 1000) + 300;

const url = cloudinary.utils.private_download_url(doc.publicId, doc.format,

{ resource_type: doc.resourceType || 'image', type: 'authenticated', expires_at: expiresAt

});

return { url, expiresAt };

});

WATCH OUT · Why not a normal signed delivery URL? Cloudinary's signed delivery URLs for `authenticated` assets **do not expire**. Expiring CDN tokens need a paid plan. The `private_download_url` (download API) checks `expires_at` on the server, so it works on the free plan. Store the `public_id` in Firestore, never a URL. A URL is a capability, and anyone who copies it can use it.

6.4 Client usage

```
// src/services/api.js - the ONLY place the frontend talks to the trusted API
import { auth } from './firebase';
const BASE = import.meta.env.VITE_API_BASE_URL;
export async function api(path, { method = 'GET', body } = {}) {
  const token = await auth.currentUser?.getIdToken();
  let res;
  try {
    res = await fetch(`${BASE}${path}`, { method, headers: {
      'Content-Type': 'application/json', ...(token && { Authorization: `Bearer ${token}` }) },
      body: body && JSON.stringify(body) });
  } catch { throw Object.assign(new Error('Service unavailable. Try again shortly.'), { code: 'UNAVAILABLE' }); }
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw Object.assign(new Error(data.error?.message || 'Request failed'), { code: data.error?.code, status: res.status });
  return data;
}
export const confirmBooking = (id) => api(`/api/bookings/${id}/confirm`, { method: 'POST' });
```

7. Core algorithms

7.1 Booking overlap

- Represent each booking as a **half-open interval** `[start, end)`. Overlap $\Leftrightarrow a.start < b.end \ \&\& \ b.start < a.end$. Adjacent stays (check-out = next check-in) do **not** overlap (TC-BOOK-009).
- **Client pre-check** at request time (fast feedback only). **Server check** at confirm time is authoritative.
- Pending requests never block each other (TC-BOOK-011).
- Short-term rentals use dates. Viewing appointments on long-term or sale listings use time slots. Store both as Timestamps so the same function works.

7.2 Trust & Fairness Score (GitHub Actions, Python)

Formula and rules: see *01 Requirements §8*. Implementation notes:

- **Median, done right:** for an even number of comparables, average the two middle values. The old plan used `arr[floor(n/2)]`, which is biased upward.
- Comparables query: verified, same city and type, same `purpose` and `pricePeriod`, floor area within $\pm 20\%$, excluding the listing itself. With fewer than 3, the label is **Insufficient data**.
- Write results with a **batched write** (max 500 operations per batch) to save quota.
- Run **daily** (cron `0 18 * * *` UTC = 02:00 Manila), with the expired-booking completion job first.

7.3 Loan / amortization

```
P = price - downPayment;  r = annualRate / 12;  n = years * 12
monthly = (r == 0) ? P / n : P * r(1+r)^n / ((1+r)^n - 1)
```

Guard `r = 0` (TC-LOAN-004), reject a negative or zero term, and round only for display.

7.4 Bank catalog upsert

The document ID is `${bank}_${referenceNo}`, written with `set(..., merge=True)`. That makes upserts idempotent. Items missing from a run are marked `delisted`, not deleted. Only **Metrobank** is enabled. The Landbank and BDO adapters stay disabled until their selectors and API shapes are verified against live pages.

8. Notifications design

- Recipient model: `recipientId = uid` for personal feeds, `"__admins__"` for the shared admin feed (disputes and listing submissions only).
- **Atomic where it matters:** a listing submission and its admin alert go in one batch (FR-026). If the alert is rejected, the submission fails visibly.
- **Best-effort elsewhere:** booking requests, decisions and reviews never fail because a notification failed. Log and continue.
- Admin read state is shared, because all admins read one `__admins__` feed. This is a documented limitation; per-admin read receipts are a future item.

9. Coding standards (all members, all AI output)

Area	Standard
Branches	<code>features/<version>-<topic></code> → PR into <code>dev</code> → release PR <code>dev</code> → <code>main</code> (see <i>06 Roadmap</i>)
Commits	Conventional Commits: <code>feat:</code> , <code>fix:</code> , <code>docs:</code> , <code>test:</code> , <code>chore:</code>
Reviews	At least one human reviewer. AI-generated code touching rules, API, overlap or trust logic needs two
Config	All keys in <code>.env</code> / host env vars. Commit <code>.env.example</code> with names only
Errors	Show the user what happened and what to do next. Log the technical detail without PII
Tests	Every bug fix adds a regression test. Pure logic has unit tests. Rules have emulator tests
Dependencies	<code>npm audit</code> must be clean before release. Pin exact versions for <code>firebase-admin</code> , <code>cloudinary</code>
AI assistants	Paste the relevant section of these documents into the prompt. Never paste secrets, service-account JSON or real user data